

Curso: Engenharia de Software

Período: 6º período

Discentes: Jabes Junio Souza Protazio; Pedro Augusto Rios Alves.

Disciplina: Sistemas Distribuídos

Atividade prática sobre middlewares

1) Identificação e justificativa da plataforma

- **Nome da plataforma escolhida:** Java RMI
- **Categoria de middleware:** Objetos distribuídos / RPC
- **Modelo de comunicação adotado:** Invocação remota de métodos
- **Principais abstrações oferecidas:** Suas principais abstrações são a interface remota, o objeto remoto e o RMI Registry para localização dos serviços
- **Motivo da escolha:** A escolha se justifica pela integração nativa com Java, ausência de dependências externas e facilidade para demonstrar comunicação entre processos.
- **Exemplos de cenários reais em que essa plataforma pode ser utilizada:** Aplicações corporativas distribuídas escritas em Java

2) Preparação e execução do ambiente

- **Versão da plataforma:** Tanto o cliente quanto o servidor utilizam o JDK 21.
- **Principais parâmetros de configuração:** Não foram necessários parâmetros de configuração específicos para a aplicação desenvolvida.
- **Portas e endpoints utilizados:**
 - Porta do RMI Registry: 1099
 - Endpoint: //localhost/rmi
- **Evidências de execução:** Na pasta de evidências no projeto

3) Concorrência ou múltiplas instâncias

- **As requisições ou mensagens são distribuídas entre as instâncias?** Não há distribuição entre diferentes instâncias do servidor, pois a aplicação possui apenas um servidor RMI. Cada cliente envia sua própria requisição para o mesmo objeto remoto.
- **Há balanceamento ou competição por mensagens?** Não. O Java RMI utilizado nessa aplicação não realiza balanceamento de carga entre servidores e também não existe competição por mensagens, como ocorreria em um middleware baseado em filas.
- **Uma mesma informação é entregue a uma ou a várias instâncias?** Cada chamada pertence ao cliente que a realizou. O resultado de uma função é devolvido somente ao cliente responsável pela requisição, não sendo distribuído para os demais clientes.
- **Qual mecanismo da plataforma determina esse comportamento?** O modelo de chamada de procedimento remoto ponto a ponto do RMI, pois o cliente consulta o RMI Registry apenas para localizar o endereço (IP e porta) do servidor. Assim que obtém essa referência, o cliente se conecta diretamente àquele servidor específico, faz a chamada e fica bloqueado aguardando a resposta.

4) Experimento de falha

- **O que acontece com requisições, mensagens ou eventos durante a falha? A informação é perdida, armazenada, reenviada ou rejeitada? Os demais componentes continuam funcionando?** Se o servidor parar de funcionar, as novas requisições são rejeitadas. Entretanto, se um dos clientes parar de funcionar, as requisições de outros clientes não são afetadas.
- **O middleware detecta a indisponibilidade?** O servidor não sabe sobre a indisponibilidade do cliente, mas o cliente sabe sobre a indisponibilidade do servidor.
- **O que acontece quando o componente é iniciado novamente?** Quando o servidor é iniciado novamente, as requisições voltam a ser recebidas, processadas e devolvidas para os clientes.
- **Evidências de falha:** Na pasta de evidências no projeto

5) O que o middleware realizou para a aplicação que precisaria ser implementado manualmente caso essa plataforma não fosse utilizada? O Java RMI abstraiu a comunicação entre cliente e servidor, permitindo que o cliente chamasse os métodos remotos sem precisar implementar manualmente sockets, troca de mensagens, serialização dos dados, localização do serviço e tratamento da comunicação de rede. Sem o middleware, esses mecanismos teriam que ser desenvolvidos diretamente pela aplicação.

6) Questões para análise

- (a) **Qual é o papel do middleware na solução implementada?** O Java RMI atua como intermediário entre cliente e servidor, permitindo a chamada de métodos remotos e abstraindo detalhes da comunicação entre os processos.

- (b) **Qual categoria melhor descreve a plataforma escolhida? Justifique.** A plataforma pertence à categoria de Objetos Distribuídos / RPC, pois permite que o cliente invoque métodos de um objeto localizado em outro processo.
- (c) **O modelo de comunicação utilizado é síncrono, assíncrono ou híbrido?** O modelo é síncrono, pois o cliente aguarda o servidor executar o método remoto e retornar uma resposta antes de continuar a execução.
- (d) **Os componentes precisam conhecer diretamente a implementação uns dos outros?** Não. O cliente precisa conhecer apenas a interface remota, sem precisar conhecer a classe que implementa o serviço.
- (e) **Existe desacoplamento espacial, temporal ou ambos? Explique.** Existe um desacoplamento espacial, pois o cliente acessa o serviço por meio de uma referência remota e não conhece sua implementação. Entretanto, não há desacoplamento temporal, pois cliente e servidor precisam estar disponíveis ao mesmo tempo para que a chamada seja realizada.
- (f) **Quais detalhes de rede ou infraestrutura foram abstraídos pelo middleware?** O RMI abstraiu o uso direto de sockets, a transmissão e serialização dos dados, a localização do objeto remoto e a comunicação da resposta entre cliente e servidor.
- (g) **Como a plataforma lida com múltiplas instâncias ou concorrência?** O Java RMI permite que vários clientes façam chamadas ao mesmo objeto remoto, atendendo requisições concorrentes.
- (h) **Como o sistema se comportou durante o experimento de falha?** Ao interromper o servidor, o cliente deixou de conseguir realizar a chamada remota e recebeu uma exceção de comunicação. Após o servidor ser iniciado novamente, novas chamadas puderam ser realizadas normalmente.
- (i) **Quais mecanismos da plataforma contribuem para escalabilidade?** O RMI permite atender múltiplos clientes e distribuir componentes em máquinas diferentes.
- (j) **Quais mecanismos contribuem para tolerância a falhas?** O principal mecanismo é a detecção de falhas de comunicação por meio de exceções, como o RemoteException.
- (k) **Quais limitações ou desvantagens foram identificadas?** Entre as principais limitações estão a dependência do Java, o modelo predominantemente síncrono e a ausência de mecanismos nativos de persistência de mensagens, balanceamento de carga e recuperação automática de falhas.
- (l) **Como seria a implementação da mesma solução sem o middleware?** Sem o RMI, seria necessário utilizar sockets diretamente e implementar manualmente a conexão, o envio e recebimento de dados, a serialização dos parâmetros, a identificação das operações e o tratamento das respostas.

- (m) **Em quais cenários reais a plataforma escolhida seria adequada?** Aplicações corporativas distribuídas escritas em Java
- (n) **Em quais cenários você não recomendaria essa plataforma?** O Java RMI não é indicado quando é necessária integração entre diferentes linguagens, comunicação assíncrona, grande escalabilidade, alta tolerância a falhas ou arquiteturas modernas baseadas em eventos e microsserviços.

7) Síntese comparativa

- (a) **Plataforma** Java RMI
- (b) **Categoria** Objetos distribuídos / RPC
- (c) **Modelo** Invocação remota síncrona de métodos
- (d) **Broker?** Não
- (e) **Persistência** Não possui persistência de requisições
- (f) **Escalabilidade** Permite múltiplos clientes, mas não oferece balanceamento automático de carga
- (g) **Tolerância a falhas** Limitada; detecta falhas por exceções, mas não possui recuperação automática